# MediCore · Informe final

Corte: 2026-11-06, rama `Dev_Diego`. Los resultados son de la corrida 9 de Jenkins, salvo donde se indica otra fuente.

## Conclusión

**Decisión propuesta: NO-GO para producción y GO condicionado para QA.**

Todos los criterios automáticos se cumplieron: la agenda resistió la carga sin doble reserva y una caída de RabbitMQ no perdió datos clínicos. Aun así, MediCore no debe pasar a producción: faltan entregables exigidos y nunca se desplegó en Azure.

La decisión final corresponde al equipo.

## Métricas

| Área | Herramienta | Resultado | Criterio | Estado |
|---|---|---|---|---|
| Unitarias e integración | xUnit | 304 de 304 | 100 % | Cumple |
| API | Insomnia | 61 solicitudes, 65 de 65 comprobaciones | 100 % | Cumple |
| Web | Cypress, Chrome | 116 de 116 | 100 % | Cumple |
| Móvil | Maestro | 8 de 8 con el código final, en emulador contra QA | 5 a 8 flujos | Cumple |
| Calidad | SonarQube | Puerta aprobada, 0 bugs, 74,2 % de cobertura | Puerta aprobada | Cumple |
| SAST | Fortify | No ejecutado, sin licencia | Exigido | **Pendiente** |
| SCA | npm audit y NuGet | 0 vulnerables | 0 críticas | Cumple |
| SCA | Dependency-Check | 4 críticas, todas falsos positivos justificados | 0 sin aceptar | Cumple |
| DAST | ZAP web y API con 5 roles | 0 alertas altas, 0 errores 500, 40 de 45 operaciones alcanzadas | 0 altas sin aceptar | Cumple |
| Rendimiento | k6 | Umbrales cumplidos | Declarados antes | Cumple |
| Accesibilidad | axe-core | 14 pantallas, 0 infracciones | 0 | Cumple |
| Usabilidad | Prueba con personas | No aplicada | 5 participantes | **Pendiente** |

| Área | Herramienta | Resultado | Criterio | Estado |
|---|---|---|---|---|
| Pipeline | Jenkins | Corrida 9: 18 etapas completas, con criterios de salida cumplidos y aprobación humana en la etapa 17 | Bloquea si falla | Cumple |
| Infraestructura | Terraform y Ansible sin Azure | Terraform 3 de 3 con proveedor simulado; Ansible sin fallas en Ubuntu desechable | Reproducible | Cumple en simulación |
| Nube | Azure | Sin desplegar: el equipo decidió no usar recursos de pago | Exigido | **Pendiente** |

### Rendimiento

| Perfil | Peticiones | req/s | p95 lectura | p95 reserva | Error inesperado | Reservas | 409 |
|---|---|---|---|---|---|---|---|
| Load, corrida 5 | 20 399 | 75,2 | 5,3 ms | 8,9 ms | 0 % | 3260 | 41 |
| Stress, corrida manual | 41 620 | 196,4 | 6,2 ms | 8,6 ms | 0 % | 6672 | 186 |
| Spike, corrida manual | 43 478 | 328,0 | 7,4 ms | 11,5 ms | 0 % | 6954 | 188 |

Después de Stress y Spike hubo 0 solapes, 0 eventos pendientes y 0 mensajes en la DLQ. Se ejecutaron antes de los últimos cambios, que no tocaron agenda ni mensajería.

No se alcanzó el punto de quiebre. k6, la API y la base de datos compartieron el equipo, así que estas cifras no representan Azure.

### Defectos

Se registraron 26 defectos: 23 cerrados y 3 abiertos, ninguno crítico ni de severidad alta. Todos están en GitHub Issues. Las pruebas de esta etapa encontraron defectos reales que las suites anteriores no veían:

- **k6:** el outbox acumuló 14 405 eventos pendientes después del pico.
- **ZAP:** un carácter nulo provocaba un error 500.
- **SonarQube:** rechazó código nuevo por duplicación y cobertura.
- **axe:** el calendario no cumplía el contraste.
- **Reinicio de Docker:** QA no volvía a arrancar si Jenkins lo había creado.
- **Ansible:** el playbook no instalaba un paquete que necesitaba.

## Escenario crítico

| Pregunta del documento | Resultado | Evidencia |
|---|---|---|
| ¿Hay doble reserva con alta demanda? | No: 0 solapes por médico y por paciente | `reports/k6/integridad.txt`, `AppointmentsApiTests` |

| Pregunta del documento | Resultado | Evidencia |
|---|---|---|
| ¿Hay consistencia mientras médicos consultan? | Sí: lecturas y reservas simultáneas sin errores inesperados | `reports/k6/` |
| ¿Se pierde un resultado si cae RabbitMQ? | No: responde 201 y la notificación queda pendiente | `documentacion/evidencias/resiliencia-qa.md` |
| ¿Se recupera y evita duplicados? | Sí: se envía al volver y el paciente recibe una sola | `documentacion/evidencias/resiliencia-qa.md` |

## Flujo asíncrono de notificaciones

1. El resultado y su evento `LabResultReady` se guardan en la misma transacción de PostgreSQL.
2. El Worker publica el evento en RabbitMQ con confirmación del broker.
3. El consumidor crea la notificación y confirma el mensaje.
4. Cada notificación emite `PatientNotification`, que la marca como despachada.

Con el Worker o RabbitMQ detenidos, en QA se comprobó lo siguiente:

- La API respondió 201 y la notificación quedó `Pending`.
- Al volver el servicio pasó a `Sent`.
- El paciente recibió exactamente una notificación.

Con Redis detenido, el inicio de sesión respondió 503 y el token activo 401. Volvió a funcionar al iniciar Redis.

## Respaldos

PostgreSQL se respalda cada 6 horas, con cifrado y verificación. En un simulacro se destruyeron la base y el volumen de respaldos, y se restauró desde una copia exportada: 14 tablas idénticas, 36 849 filas y 72 s de recuperación en este equipo. La pérdida máxima esperada es de 6 horas. Las copias siguen en el mismo disco salvo exportación manual. Detalle: evidencias/respaldos.md.

## Pipeline

| Corrida | Resultado | Qué demostró |
|---|---|---|
| 2 | Falló | SonarQube rechazó el código nuevo y Dependency-Check no pudo abrir su base |
| 3 | Falló | El smoke no llegaba a QA desde el contenedor de Jenkins |
| 4 | Falló | La colección de API falló porque QA arrastraba datos de corridas anteriores |
| 5 | Inestable | Las 18 etapas llegaron al final, con Fortify y Maestro sin ejecutar |

| Corrida | Resultado | Qué demostró |
|---|---|---|
| 6 | Falló | `SIMULAR_FALLO` detuvo el pipeline: no se ejecutaron Aprobación ni Deploy final |
| 7 | Falló | SonarQube rechazó 8 incidencias nuevas y un punto de seguridad sin revisar. La puerta no se bajó |
| 8 | Falló | SonarQube rechazó 1 incidencia y otro punto de seguridad. Lo demás de la etapa 16 cumplió |
| 9 | Inestable | Puerta de SonarQube y criterios de salida cumplidos, con los respaldos incluidos. Fortify y Maestro quedan como aviso. Diego aprobó la etapa 17 y la etapa 18 registró que no hay servidor de Azure |

La corrida 5 terminó inestable a propósito: Fortify y Maestro no se ejecutaron y el pipeline no los oculta. La aprobación de la etapa 17 se dio de forma automatizada, y la etapa 18 solo registró que no hay servidor de Azure.

En la corrida 9 la aprobación la dio una persona desde Jenkins, sin automatizarla. Jenkins numera esa corrida como 3 porque perdió su historial al reiniciar Docker (HJ-04).

Las consolas están en `reports/jenkins/`.

## Por qué no es GO para producción

| Bloqueo | Por qué importa | Quién lo resuelve |
|---|---|---|
| No hay despliegue en Azure (R-18) | La ejecución sin costo no demuestra la nube real | Equipo, si decide pagar |
| Usabilidad sin aplicar | Es un entregable exigido | Equipo, con 5 personas |
| Fortify sin ejecutar | Es un entregable exigido | Equipo, con licencia |

## Riesgos residuales

| Riesgo | Nivel | Tratamiento |
|---|---|---|
| Copias en el mismo disco (R-16) | Medio | Respaldos probados. Exportarlos o llevarlos a otro lugar con `qa.sh export` |
| Ambiente de nube sin aprovisionar (R-18) | Alto | Decidir el pago y seguir `06-infraestructura.md` |
| Sin Redis no hay sesión (R-14) | Medio | Aceptado por seguridad. Vigilar el servicio |
| Servidor único (R-17) | Medio | Aceptado para QA |
| ZAP sin HTTPS real | Bajo | Repetir sobre HTTPS si se despliega |
| Puntos sensibles de Terraform | Medio | Revisar antes de aplicar |

| Riesgo | Nivel | Tratamiento |
|---|---|---|
| Avisos de Redis 2026 | Medio | Revisar y actualizar la imagen |
| Rendimiento medido en un solo equipo | Medio | Repetir en Azure |
| Contraseña de semilla débil en QA local | Bajo | `qa.sh init` genera una fuerte. Usarla antes de exponer QA |

## Criterios para cambiar la decisión

MediCore puede pasar a GO cuando se cumpla todo esto:

1. Usabilidad con 5 personas, SUS de 68 o más y 80 % de tareas sin ayuda.
2. Fortify ejecutado y sus hallazgos clasificados.
3. QA desplegado en Azure, con ZAP y k6 repetidos allí.